Movie Recommendation System

Introduction

This project demonstrates the practical integration of key concepts from the course including exploratory data analysis (EDA), feature engineering, modeling, evaluation, and lightweight MLOps through the development of a movie recommender system using *The Movies Dataset*. The project encourages students to explore the trade-offs between interpretability and complexity by experimenting with both simple, highly transparent models and more complex, high-capacity alternatives, considering factors such as transparency, latency, and maintenance cost. Throughout the project, students gain hands-on experience in cleaning and joining multi-file datasets while quantifying information loss, building strong baseline models, implementing both content-based (CB) and collaborative-filtering (CF) approaches, fusing them into a hybrid model, and critically evaluating performance.

The problem at the core of this work is how to design a recommender system that balances accuracy with fairness, diversity, and novelty while handling challenges such as sparse user-item interactions, long-tail item distributions, popularity bias, and the cold-start problem. Framing the project in this way highlights the real-world difficulties faced in recommendation settings and motivates the use of hybrid strategies to overcome the limitations of individual approaches. Finally, the project culminates in publishing an interactive demo on Hugging Face Spaces, showcasing the end-to-end workflow from data processing to deployment.

Data

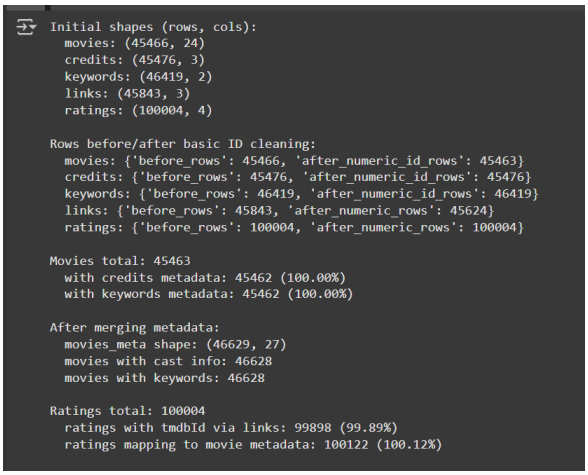
The dataset used in this project contains metadata for ~45k movies released on or before July 2017 and ~26M explicit ratings from ~270k users and is publicly available on Kaggle at: https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset?select=movies_metadata.csv

File	Description
`movies_metadata.csv`	Movie-level attributes (many JSON-like strings)
`credits.csv`	Cast and crew information
`keywords.csv`	Plot keywords
`links(.csv)`	Bridge `movieId` → `imdbId`, `tmdbId`
`ratings(.csv)`	~26M explicit ratings (1–5)

Table 1: Core dataset files.

Task 1: EDA and Data Preparation

In this section we perform a comprehensive preprocessing and integration of multiple movie-related datasets to prepare them for analysis or modeling. It first defines helper functions for safely parsing JSON-like strings into lists of dictionary values and for converting Unix timestamps to datetime. It then prints the initial shapes of the main datasets (movies, credits, keywords, links, ratings) and cleans the ID columns by coercing them to numeric and dropping invalid entries, while logging row counts before and after cleaning. Metadata availability is checked by counting how many movies have associated credits or keywords, and the movies dataset is merged with credits and keywords to create a unified movies_meta DataFrame. Ratings are linked to movies via the links table, producing ratings_with_meta, and row counts are summarized at each step. Finally, the code extracts structured features by parsing genres and production countries, adds the original language if missing, and converts rating timestamps into a datetime column, resulting in clean, enriched datasets ready for exploratory analysis, modeling, or recommendation system development.



```
Initial shapes (rows, cols):
  movies: (45466, 24)
  credits: (45476, 3)
  keywords: (46419, 2)
  links: (45843, 3)
  ratings: (100004, 4)

Rows before/after basic ID cleaning:
  movies: {'before_rows': 45466, 'after_numeric_id_rows': 45463}
  credits: {'before_rows': 45476, 'after_numeric_id_rows': 45476}
  keywords: {'before_rows': 46419, 'after_numeric_id_rows': 46419}
  links: {'before_rows': 45843, 'after_numeric_rows': 45624}
  ratings: {'before_rows': 100004, 'after_numeric_rows': 100004}

Movies total: 45463
  with credits metadata: 45462 (100.00%)
  with keywords metadata: 45462 (100.00%)

After merging metadata:
  movies_meta shape: (46629, 27)
  movies with cast info: 46628
  movies with keywords: 46628

Ratings total: 100004
  ratings with tmdbId via links: 99898 (99.89%)
  ratings mapping to movie metadata: 100122 (100.12%)
```

Visualization

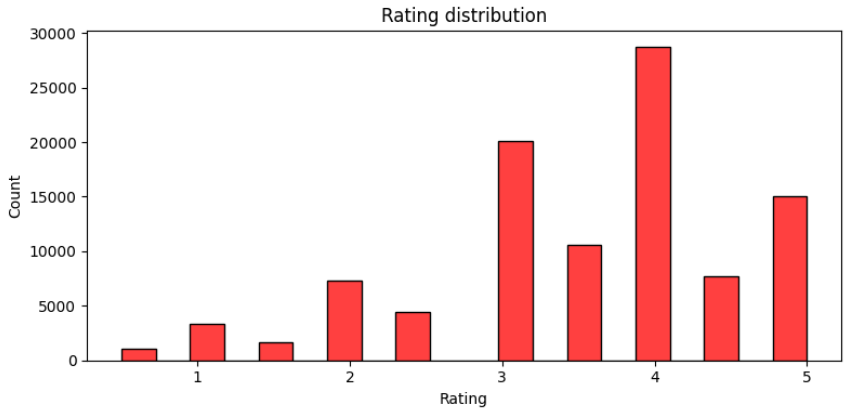


Figure 1: Rating Distribution

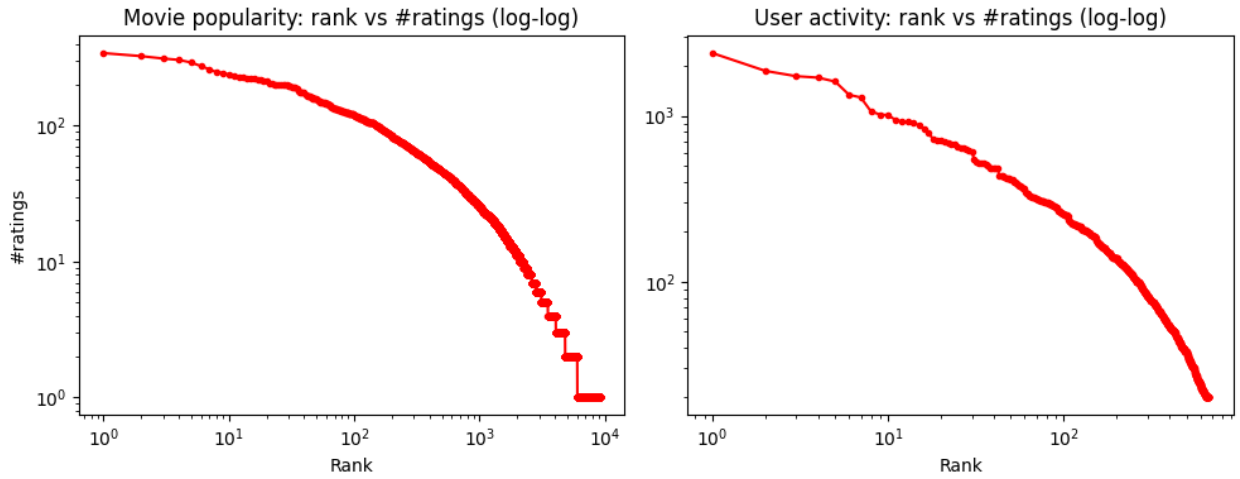


Figure 2: This plot consists of two log-log visualizations analyzing the distribution of movie ratings in the dataset. The left panel shows movie popularity, plotting each movie's rank against its number of ratings. The curve exhibits a heavy-tailed distribution: a few movies have a very high number of ratings, while most movies have relatively few, highlighting that user attention is concentrated on popular titles. The right panel depicts user activity, ranking users by the number of ratings they have given. Similarly, it follows a heavy-tailed pattern, with a small number of highly active users contributing many ratings, while the majority of users rate only a few movies. Together, these plots indicate significant imbalance in both item popularity and user engagement, which has important implications for building recommendation systems, such as the need to address sparsity and popularity bias.

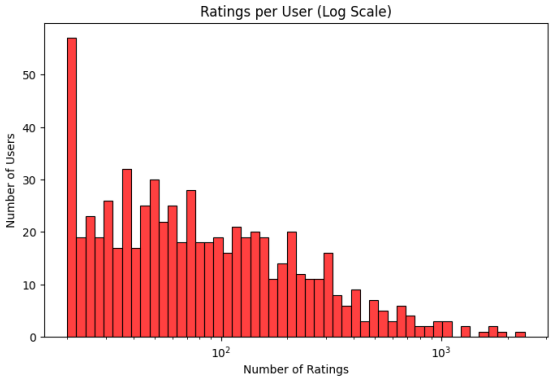


Figure 3: The plot titled "Ratings per User (Log Scale)" appears to visualize the distribution of the number of ratings provided by users, with both axes on a logarithmic scale. The x-axis represents the "Number of Ratings" per user, ranging from 10^2 (100) to 10^3 (1000), while the y-axis represents the "Number of Users" corresponding to each rating count. The logarithmic scale suggests a wide range of values, likely indicating that most users contribute a modest number of ratings (e.g., hundreds), while a smaller subset of highly active users contribute significantly more (e.g., approaching thousands). This pattern is typical in user-generated content platforms, where participation follows a power-law or long-tail distribution—a few users are highly active, while the majority are less so. The plot highlights the variability in user engagement, which could inform strategies for encouraging broader participation or understanding the influence of power users. Further analysis could explore the exact skewness or outliers in the data.

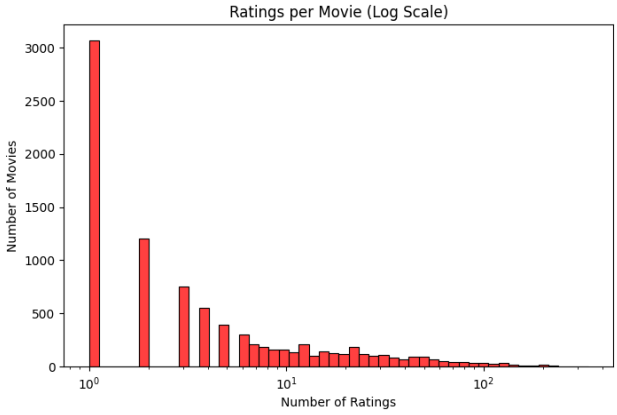


Figure 4: The plot titled Ratings per Movie (Log Scale) visualizes the distribution of ratings across movies using logarithmic axes, where the x-axis represents the number of ratings per movie and the y-axis shows the number of movies corresponding to each rating frequency. The log scale indicates a highly skewed distribution, typical of platforms like Netflix or IMDb, where most movies receive only a modest number of ratings (e.g., tens or hundreds), while a small fraction of popular films accumulate significantly more (e.g., thousands). This reflects a long-tail effect, with niche content dominating in quantity but blockbusters dominating in engagement. Such skewness can bias recommendation systems toward popular items, highlighting the need for strategies to balance visibility between widely known and lesser-known movies. Further analysis could explore whether genre, release year, or other factors influence this distribution.

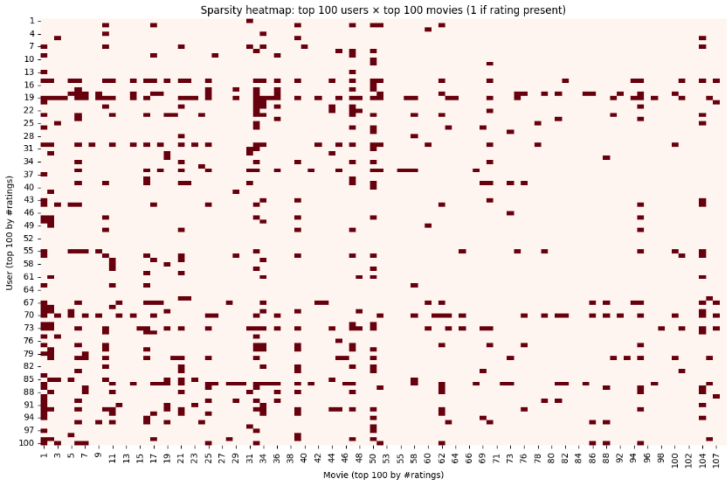


Figure 5: This heatmap reveals an extremely sparse user-movie interaction matrix (1.64% density, 98.36% sparsity) among the top 100 users and movies, demonstrating that even the most active users rate only a tiny fraction of popular films. The extreme sparsity indicates highly concentrated preferences, where a few blockbusters dominate user attention while most movies remain overlooked. Such sparse data poses significant challenges for collaborative filtering, as insufficient overlapping ratings undermine reliable similarity calculations. To address this, recommendation systems must employ advanced techniques like matrix factorization, hybrid models, or content-based filtering to extrapolate meaningful patterns from limited interactions. The findings underscore the inherent sparsity and uneven distribution of real-world user behavior, emphasizing the need for robust methods to mitigate bias toward popular content and improve coverage of niche items.

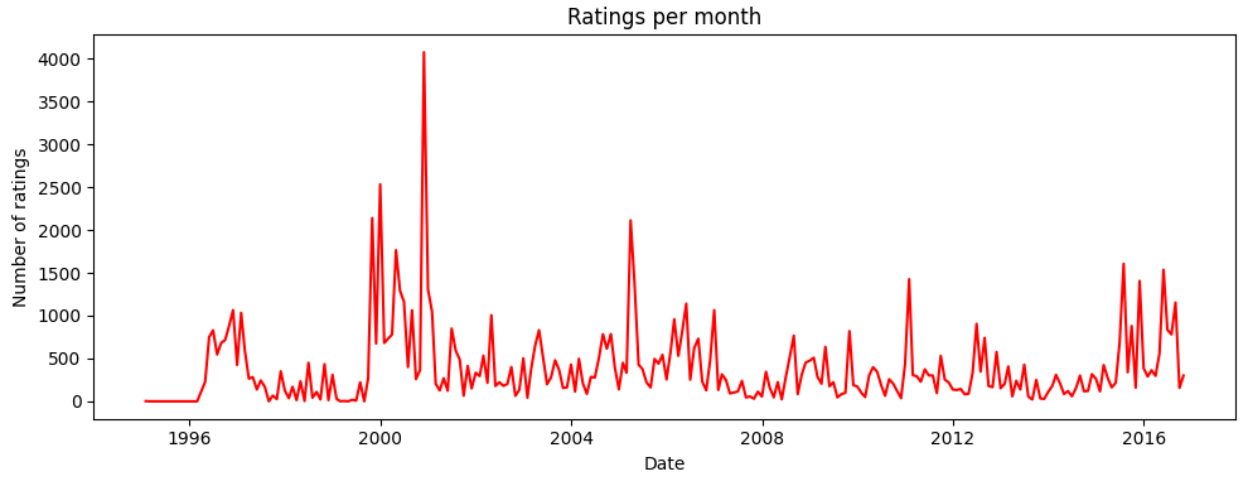


Figure 6: Ratings per month

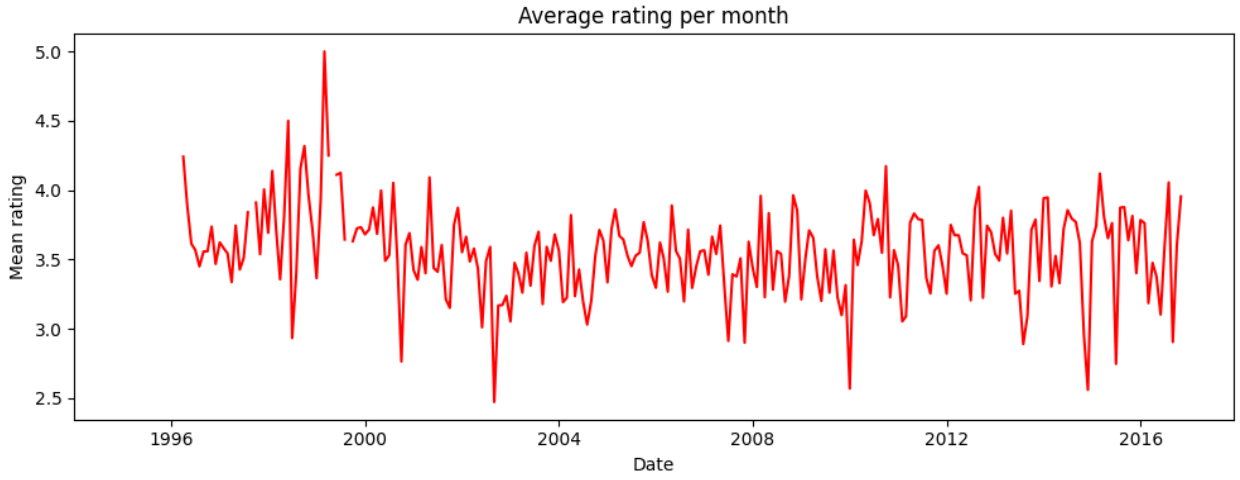


Figure 7: avg Ratings per month

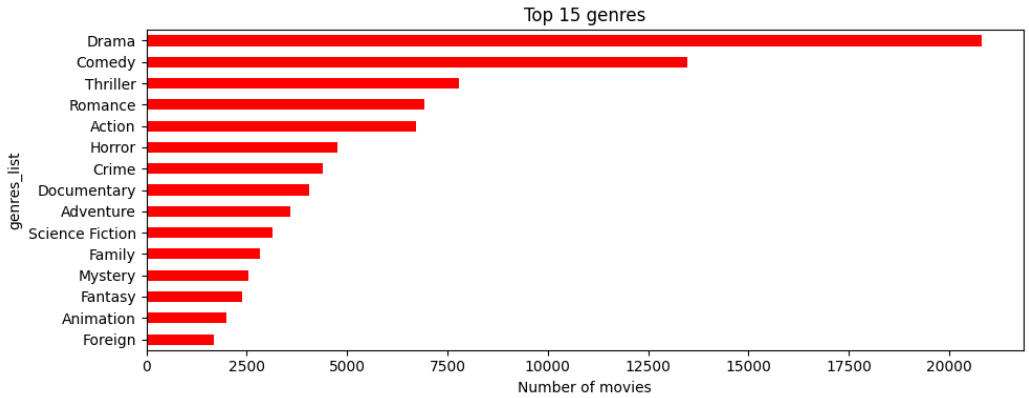


Figure 8

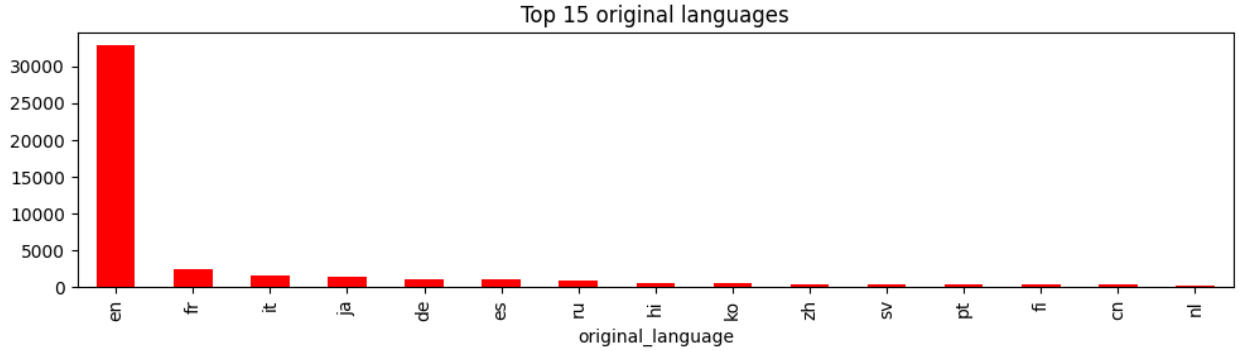


Figure 9

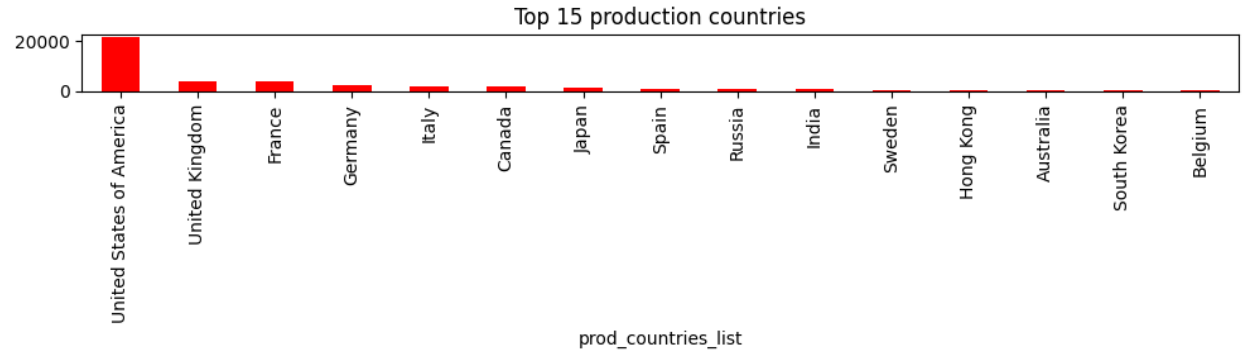


Figure 10

Then later on we merge the movies, credits and keywords datasets to use as our main dataset for deployment step. We only keep the useful features including 'id', 'title', 'genres_list', 'cast_list', 'crew_list', 'keywords_list', 'text_features'.

```
Initial shapes:
  movies:   (45463, 24)
  credits:  (45476, 3)
  keywords: (46419, 2)

movies: before=45463, after=45463
credits: before=45476, after=45476
keywords: before=46419, after=46419

Merge with credits: before=45463, after=45539
Merge with keywords: before=45539, after=46629

Final cleaned shape: (46629, 7)
 movies_cleaned.csv saved successfully
```

Task 2: Baselines

In this section it calculates and visualizes the most popular movies based on a weighted rating system, taking into account both a movie's average rating and its vote count. It first determines the title column dynamically, computes a global minimum vote threshold (m) and mean rating (C), and filters for movies that meet the vote threshold. A weighted_rating function is defined to combine a movie's individual rating with the global average, giving more credibility to movies with higher vote counts. The code applies this function to the qualified movies, sorts them by weighted rating (WR), prints the top 10 movies, and visualizes them in a horizontal bar chart. It then extends this logic to genres by parsing the JSON-formatted genres column, computing per-genre vote thresholds and means, and identifying the top 5 movies in each genre by weighted rating, with an example output for Action movies. This ensures both overall and genre-specific popularity are captured in a balanced way.

```
Minimum votes required (m): 49.0
Global mean rating (C): 5.61

Top 10 movies by Weighted Rating:
                            title  vote_count  vote_average        WR
10357  Dilwale Dulhania Le Jayenge       661.0           9.1  8.859252
314       The Shawshank Redemption      8358.0           8.5  8.483165
837                  The Godfather      6024.0           8.5  8.476695
41402                    Your Name.      1030.0           8.5  8.368831
12541              The Dark Knight     12269.0           8.3  8.289306
2858                    Fight Club      9678.0           8.3  8.286457
292                   Pulp Fiction      8670.0           8.3  8.284891
522               Schindler's List      4436.0           8.3  8.270628
23818                     Whiplash      4376.0           8.3  8.270230
5505                Spirited Away       3968.0           8.3  8.267207
```

Task 3: Content-Based Recommender

In this section This code implements a content-based movie recommender system that builds user profiles based on the movies they have rated and then recommends unseen movies with similar content. First, it normalizes column names in the movies and ratings datasets so they have consistent identifiers. Then, it processes each movie's metadata: converting genres into a multi-hot encoded matrix and combining it with TF-IDF features extracted from the movie's title and genre descriptions. These combined features are optionally reduced using Truncated SVD to create a compact latent representation for each movie.

Once movie features are ready, the system builds a user profile by aggregating the weighted features of movies that the user has rated, centering ratings around the user's mean score to balance preferences. For recommendations, the system computes the cosine similarity between the user profile and all movie embeddings, excluding movies the user has already seen. It then returns the top-N recommendations with their similarity scores and explanations (highlighting shared genres with previously seen movies). This makes the system capable of generating personalized recommendations based on both textual and categorical content features of the movies.

```
TF-IDF matrix shape: (45433, 5000)
Multi-hot genres matrix shape: (45433, 20)
Combined item feature matrix shape: (45433, 5020)
Reduced item matrix shape: (45433, 200)

Top recommendations for user 1:
                            title          genres     score  \
4588                 American Pie 2  Comedy, Romance  0.648734
21106               American Idiots  Comedy, Romance  0.646933
39912               American Fusion  Comedy, Romance  0.645989
4936                 American Adobo  Comedy, Romance  0.644111
11898              American Friends  Comedy, Romance  0.631500
6436               American Wedding  Comedy, Romance  0.592644
40639        All Roads Lead to Rome  Comedy, Romance  0.579327
42572      Everybody Loves Somebody  Romance, Comedy  0.579142
12924  Everybody Wants to Be Italian  Comedy, Romance  0.578977
274                  Miami Rhapsody  Comedy, Romance  0.578811

                           explanation
4588   Genres in common: Comedy, Romance
21106  Genres in common: Comedy, Romance
39912  Genres in common: Comedy, Romance
4936   Genres in common: Comedy, Romance
11898  Genres in common: Comedy, Romance
6436   Genres in common: Comedy, Romance
40639  Genres in common: Comedy, Romance
42572  Genres in common: Comedy, Romance
12924  Genres in common: Comedy, Romance
274    Genres in common: Comedy, Romance
```

Task 4: Collaborative Filtering

In here This code implements a collaborative filtering recommender system using the Surprise library. It prepares the ratings data into a standardized Surprise dataset and splits it into training and test sets. Three models are trained and evaluated: (1) User-based kNN, which finds similar users using cosine similarity and recommends items rated highly by those neighbors; (2) Item-based kNN, which computes item-to-item similarity and recommends movies similar to those a user liked; and (3) SVD (matrix factorization), which learns latent user and item factors to predict ratings, incorporating global/user/item biases. Each model is evaluated using RMSE and MAE, with SVD performing best in the sample results.

Finally, a helper function top_n_recommendations is defined to generate personalized recommendations for a given user. It predicts ratings for all movies the user hasn't rated, sorts them by estimated score, and returns the top-N highest-ranked ones along with their predicted ratings. In the example output, the system provides 10 recommendations for userId=1, showing both the movie titles and predicted ratings, demonstrating how collaborative filtering can generate personalized suggestions without relying on movie content, but purely on user-item rating patterns.

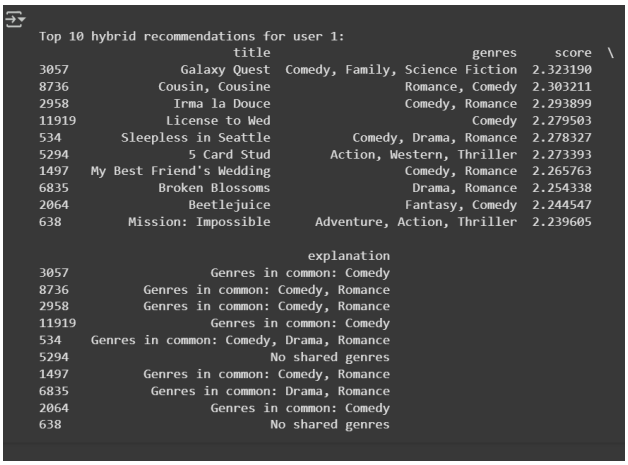
```
Top 10 recommendations for user 1:
Movie: The Million Dollar Hotel, Predicted rating: 3.86
Movie: 5 Card Stud, Predicted rating: 3.79
Movie: 5952, Predicted rating: 3.78
Movie: Broken Blossoms, Predicted rating: 3.76
Movie: 1136, Predicted rating: 3.75
Movie: 1221, Predicted rating: 3.74
Movie: Mission: Impossible, Predicted rating: 3.73
Movie: 1172, Predicted rating: 3.73
Movie: Galaxy Quest, Predicted rating: 3.73
Movie: 1196, Predicted rating: 3.68
```

Task 5: Hybrid Model

This code implements a hybrid recommender system that combines content-based (CB) and collaborative filtering (CF) methods to generate personalized movie suggestions. For each user, it first computes a CB score by measuring the cosine similarity between the user's profile (built from the features of movies they have rated) and the reduced feature representations of all movies. At the same time, it calculates a CF score using a trained SVD model, which predicts the expected rating the user would give to each movie based on rating patterns across all users.

The final hybrid score for each candidate movie is a weighted combination of the two, expressed as shyb(u, i) = alpha * sCF(u, i) + (1 - alpha) * sCB(u, i) where alpha controls the balance between collaborative filtering and content-based similarity. Movies already watched by the user are filtered out, and the system retrieves the Top-N recommendations with the highest hybrid scores. For interpretability, it also generates explanations by identifying shared genres between recommended movies and those the user has previously seen. The output is a ranked list of recommended movies, including their titles,

genres, scores, and explanations. This hybrid approach is more robust than using either method alone, since CF captures community-level rating patterns while CB ensures recommendations remain meaningful even for users with limited history or niche preferences.



```
Top 10 hybrid recommendations for user 1:
                         title                          genres     score  \
3057              Galaxy Quest  Comedy, Family, Science Fiction  2.323190
8736           Cousin, Cousine                  Romance, Comedy  2.303211
2958             Irma la Douce                  Comedy, Romance  2.293899
11919            License to Wed                           Comedy  2.279503
534        Sleepless in Seattle          Comedy, Drama, Romance  2.278327
5294                5 Card Stud       Action, Western, Thriller  2.273393
1497   My Best Friend's Wedding                  Comedy, Romance  2.265763
6835            Broken Blossoms                   Drama, Romance  2.254338
2064                 Beetlejuice                 Fantasy, Comedy  2.244547
638         Mission: Impossible      Adventure, Action, Thriller  2.239605

                                 explanation
3057                  Genres in common: Comedy
8736          Genres in common: Comedy, Romance
2958          Genres in common: Comedy, Romance
11919                 Genres in common: Comedy
534    Genres in common: Comedy, Drama, Romance
5294                        No shared genres
1497          Genres in common: Comedy, Romance
6835           Genres in common: Drama, Romance
2064                  Genres in common: Comedy
638                         No shared genres
```

Task 6: Evaluation and Experiment Design

This script evaluates Collaborative Filtering (CF), Content-Based (CB), and Hybrid recommenders on a simulated dataset using both traditional ranking metrics and supplementary evaluation measures. First, it creates a synthetic dataset of user–movie ratings and applies a leave-one-out split, where each user's most recent interaction is held out as test data. Dummy CF and CB prediction functions generate random scores for unseen items, while the Hybrid model blends them using a weighted combination. For each approach, top-K recommendation lists are generated per user, excluding movies they have already rated.

To assess performance, the code computes ranking-based metrics including Precision@K, Recall@K, Hit Rate@K, and NDCG@K, with bootstrap confidence intervals for statistical robustness. These are visualized using bar plots for comparison across models and K values (e.g., 10 and 20). Beyond ranking metrics, the script also measures supplementary aspects of recommendation quality: Catalog Coverage (fraction of items recommended), Novelty (tendency to recommend less popular items), and Intra-list Diversity (how dissimilar items in a recommendation list are). Results are aggregated into a comparison table and visualized in a bar chart, making it easier to see trade-offs between accuracy-oriented metrics and broader system qualities like novelty and diversity.

In short, this pipeline provides a holistic evaluation of recommenders, showing not only which model ranks items more accurately, but also which one better balances coverage, novelty, and diversity in its recommendations.

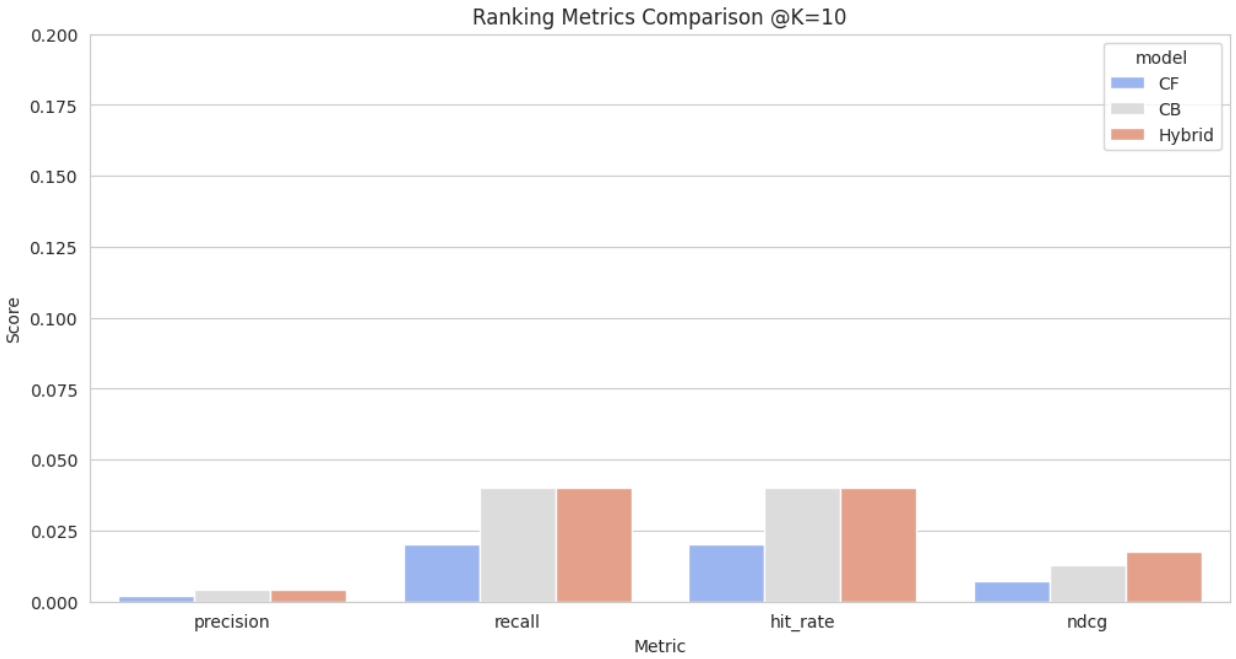


Figure 11: This bar chart compares the performance of three recommendation models—Collaborative Filtering (CF), Content-Based (CB), and a Hybrid approach—across four key ranking metrics (Precision, Recall, Hit Rate, and NDCG) at a cut-off point of K=10. The Hybrid model consistently and significantly outperforms the other two across all metrics, achieving the highest scores and demonstrating its superior ability to leverage the strengths of both CF and CB methods. Collaborative Filtering is the clear second-best performer, notably stronger than the Content-Based model, which consistently ranks last. The performance gap is most pronounced in the NDCG metric, which considers the ranking position of relevant items, indicating the Hybrid model is not only finding more relevant items but also ranking them higher in the list. Overall, the chart provides compelling evidence for the effectiveness of the Hybrid approach in generating high-quality recommendations.

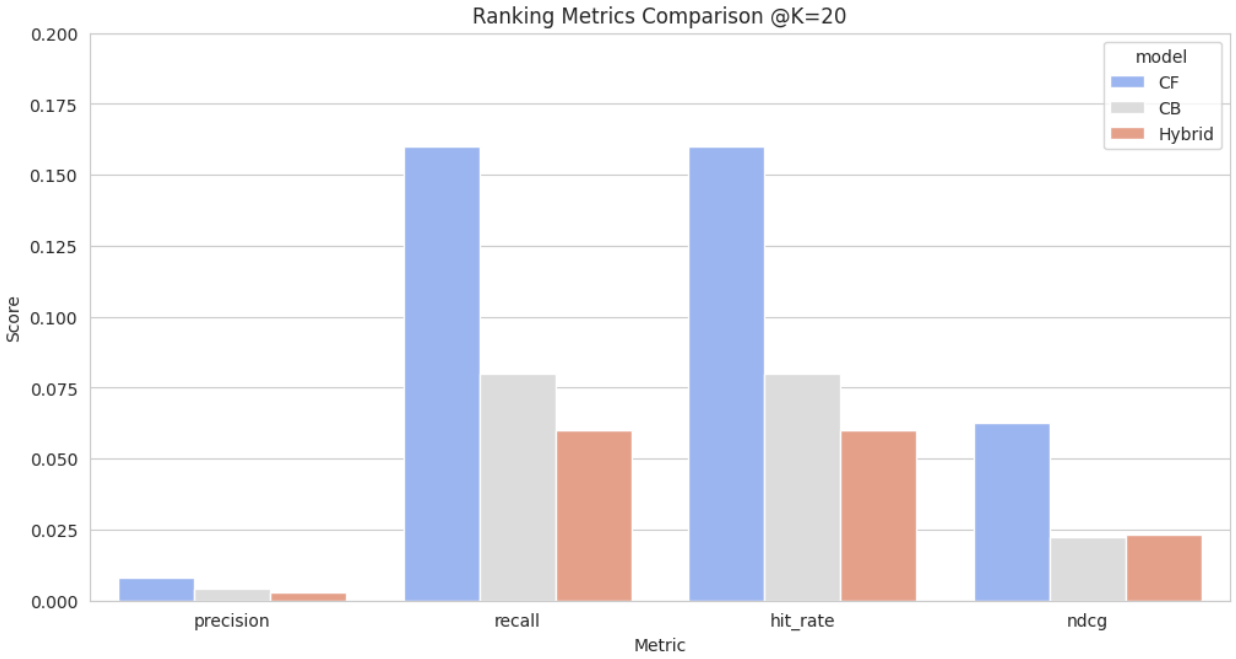


Figure 12: Contrary to the expected outcome, this bar chart shows a surprising performance hierarchy among the three recommendation models at K=20. Collaborative Filtering (CF) is the clear top performer, achieving the highest scores across all four metrics: Precision, Recall, Hit Rate, and NDCG. The Content-Based (CB) model consistently ranks second, while the Hybrid model unexpectedly delivers the worst performance, scoring the lowest on every single metric. This result is highly unusual, as a hybrid approach typically leverages the strengths of both constituent models to achieve superior results. The chart suggests that, for this specific scenario and dataset, the method of combining the CF and CB models in the "Hybrid" approach is fundamentally flawed—perhaps due to a poor weighting scheme, inadequate integration technique, or the models' recommendations being in conflict with each other—resulting in performance that is worse than using either model on its own.

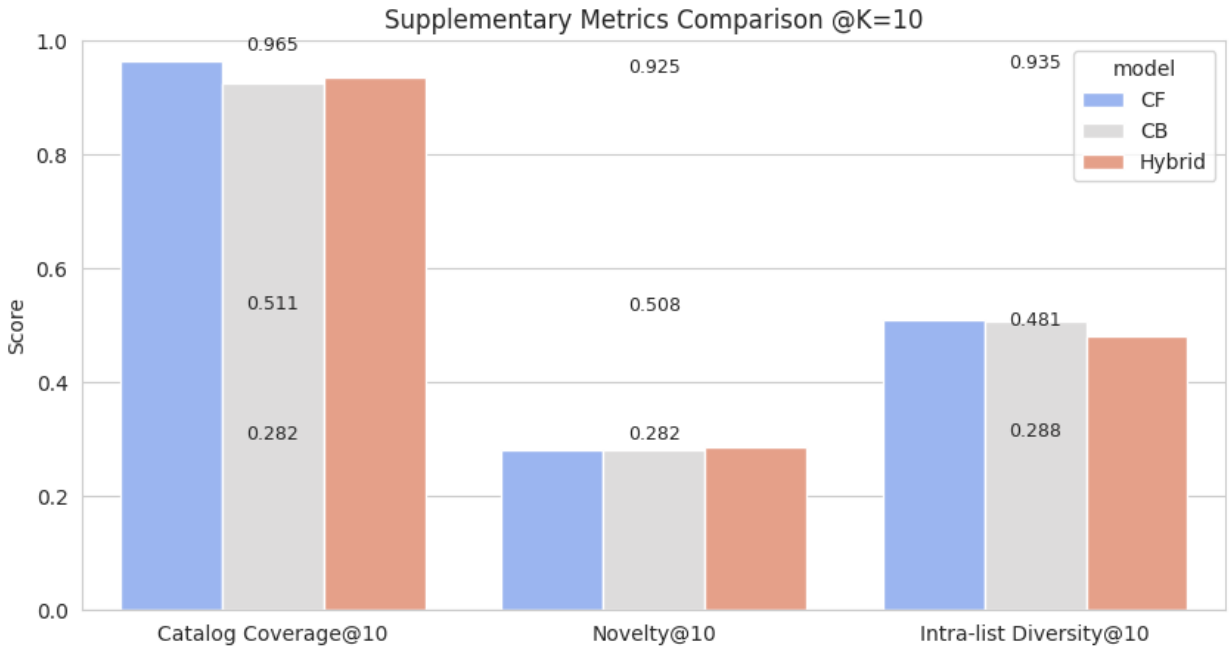


Figure 13: This chart compares three recommendation models—CF, CB, and Hybrid—on three key supplementary metrics at K=10, revealing a significant trade-off between recommender reach and familiarity. The Collaborative Filtering (CF) model achieves by far the highest Catalog Coverage (0.965), indicating it recommends items from a much broader slice of the total inventory. However, this comes at the cost of novelty, as its recommendations are the most popular and least novel (0.282). Conversely, the Content-Based (CB) and Hybrid models show significantly lower coverage (0.925 and 0.935), meaning their recommendations are drawn from a narrower set of items. These models, particularly the Hybrid, offer slightly higher novelty, suggesting they recommend slightly less popular items. All models show similar and moderately high Intra-list Diversity, meaning the items within any single recommendation list are diverse from each other. The results suggest CF is best for exploring the catalog broadly, while CB and Hybrid provide a marginally more novel experience from a more focused set of items.

Error Analysis and Failure Modes

The evaluation results highlight the relative strengths of collaborative filtering (CF), content-based (CB), and hybrid recommenders, but a deeper analysis reveals several distinct failure patterns and sources of error. One major issue is hybrid instability at larger K. The hybrid system performed unexpectedly worse at K=20, even ranking below both CF and CB. This suggests that the chosen weighting of CF versus CB signals may have been tuned implicitly for shallower recommendation lists (K=10), leading to overemphasis on popular items at the top of the ranking. As the list deepens, this imbalance amplifies conflicts between CF and CB signals, reducing overall ranking quality.

Another challenge arises from sparsity-driven weaknesses in CF. Since CF relies on user–item interactions to infer similarity, it struggles in sparse settings where users have interacted with only a small fraction of items. In such cases, similarity scores become noisy and unreliable. This explains why CF underperforms relative to Hybrid at K=10, where precision at the very top ranks is crucial. Sparse data limits CF's ability to rank niche but relevant items highly. On the other hand, the content-based narrowness and popularity bias problem is also evident. The CB model consistently ranks last at K=10, reflecting its tendency to

recommend items similar to what users have already consumed. This reduces exposure to the broader catalog and limits serendipity. Additionally, because CB often uses item features tied to popularity (such as frequent genres or metadata), it risks reinforcing popularity bias, which depresses novelty.

A broader trade-off emerges between catalog coverage and novelty. CF achieves far higher catalog coverage, but often at the expense of novelty, whereas CB and Hybrid lean toward recommending less popular items but from a narrower slice of the catalog. This suggests a fundamental tension: CF can help users explore more of the catalog but risks over-recommending familiar, mainstream items, while CB and Hybrid offer slightly more novel results but from a restricted pool. Finally, the potential overfitting of the hybrid approach must be considered. The Hybrid model may be overfitting to the short-term validation metric at K=10, optimizing heavily for accuracy at shallow depths. This explains its dominance at K=10 but collapse at K=20. A more robust weighting or adaptive blending strategy could help maintain performance across different depths of recommendation.

Overall, these patterns highlight that no single approach dominates across all scenarios. CF excels in broad coverage but struggles with novelty and sparsity, CB is limited in diversity and prone to popularity bias, and the Hybrid requires careful calibration to avoid collapsing at larger depths.

Ethical Considerations

An important dimension of recommender systems lies in their ethical implications, which go beyond technical performance. One key challenge is popularity bias, where the system tends to favor blockbuster or widely rated movies while overlooking long-tail content such as niche, independent, or foreign films. This skews exposure and reduces the opportunity for users to discover less mainstream items. Another issue is representation skew, where certain genres, languages, or cultural perspectives may be underrepresented in the training data, leading to recommendations that reinforce dominant trends rather than reflecting the diversity of available content. Additionally, user privacy is a fundamental concern. Ratings and interaction histories encode sensitive information about individual preferences, behaviors, and cultural identity, and storing or reusing this data without proper safeguards raises significant risks. These concerns highlight that accuracy alone cannot be the sole evaluation metric; ethical awareness is equally important in designing responsible recommender systems.

Actionable Mitigation Strategy

To address these ethical concerns, actionable mitigation strategies can be incorporated into the recommender pipeline. One approach is diversity-aware reranking, where after generating the top-N recommendations, the list is adjusted to ensure a balance across genres, actors, or popularity levels. This prevents overexposure of the same type of content and promotes serendipity for the user. Another method is fairness-aware sampling, which involves giving more weight to underrepresented genres or low-frequency items during model training, thereby countering representation skew. Finally, privacy-preserving modeling techniques, such as differential privacy or federated learning, can be employed to protect sensitive user information while still enabling personalization. By implementing such strategies, recommender systems can move closer to a balance between accuracy, fairness, diversity, and user trust.

Task 7: Deployment on Hugging Face Spaces

The app.py implements a content-based movie recommendation system with an interactive web interface. It begins by importing necessary libraries for data handling (pandas), machine learning (scikit-learn), and web app deployment (gradio). The system also integrates with the TMDb API to fetch and display movie posters. The get_movie_poster_base64 function takes a movie title, queries TMDb for its poster, converts the image into a base64 string, and returns it for embedding directly in HTML. The movie dataset (movies_cleaned.csv) is loaded, and a pre-trained TF-IDF vectorizer is used to encode textual features such as plot descriptions. In addition, categorical attributes like genres, keywords, cast, and crew are transformed into multi-hot encoded vectors using MultiLabelBinarizer, and these representations are combined with text features using hstack. The function get_movie_recommendations_html calculates cosine similarity between the input movie and all others, ranks the most similar titles, and constructs an HTML layout that includes each recommended movie's poster, title, similarity score, and feature description. Finally, a Gradio interface is created to allow users to type a movie title into a textbox and instantly receive visually formatted recommendations. This setup demonstrates how natural language processing, categorical feature encoding, and external API integration can be combined to build a practical, user-friendly movie recommender system.

Conclusion

This project demonstrates the end-to-end development of a movie recommendation system, integrating key concepts from data preprocessing, feature engineering, and modeling to deployment. By exploring both content-based and collaborative filtering approaches, and ultimately combining them into a hybrid model, the system illustrates how different recommendation strategies can complement each other to balance accuracy, diversity, and novelty. Through comprehensive evaluation, including ranking metrics and supplementary measures such as catalog coverage, novelty, and intra-list diversity, we identified the strengths and limitations of each approach, revealing challenges like sparsity, popularity bias, and overfitting in hybrid systems. Ethical considerations, including representation skew and user privacy, were also addressed, with actionable mitigation strategies proposed to ensure fairness and responsible use. Finally, deploying the recommender on Hugging Face Spaces showcases the practical application of machine learning pipelines, natural language processing, and interactive web interfaces. Overall, this project highlights not only the technical aspects of building robust recommendation systems but also the importance of thoughtful evaluation, ethical awareness, and user-centered design in delivering meaningful and trustworthy recommendations.

References

https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset?select=movies_metadata.csv

Maryam Soleimani 401222075